

Appendix A: Technical Execution Log

Subject: End-to-End Pipeline for Training CogVideoX-5B LoRA

Target Hardware: Single NVIDIA RTX A6000 (48GB VRAM)

1. Local Data Curation (The "Auto-Director")

Instead of manual editing, we utilized a Python script integrating **Google Gemini 3 Flash** to semantically analyze raw video and **FFmpeg** to extract high-quality samples.

- **Objective:** Create a dataset of 2-4 second high-action clips.
- **Logic:** The script uploads video to Gemini, requests timestamps of "high energy" moments, and performs frame-accurate cuts.

Script: `prepare_dataset.py`

```
import os
import time
import json
import google.generativeai as genai
import ffmpeg

# ===== CONFIGURATION =====
# 1. PASTE YOUR API KEY HERE
API_KEY = ""

# 2. FOLDER SETUP
# Get the directory where this script is currently running
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))

# INPUT: Look for videos in the SAME folder as this script
INPUT_FOLDER = CURRENT_DIR

# OUTPUT: Create a subfolder inside the current directory
OUTPUT_FOLDER = os.path.join(CURRENT_DIR, "mr_beast_dataset")
# =====

# Configure Google AI
genai.configure(api_key=API_KEY)
```

```

def analyze_and_cut(video_path, filename):
    print(f"Processing: {filename}...")

    try:
        # --- STEP 1: UPLOAD TO GOOGLE ---
        print("  [1/3] Uploading video to Gemini...")
        video_file = genai.upload_file(path=video_path)

        # Wait for processing to complete
        while video_file.state.name == "PROCESSING":
            time.sleep(2)
            video_file = genai.get_file(video_file.name)

        if video_file.state.name == "FAILED":
            print(f"  ERROR: Google failed to process video. State: {video_file.state.name}")
            return

        # --- STEP 2: ANALYZE VIDEO ---
        print("  [2/3] Analyzing with Gemini 3 Flash...")

        prompt = """
        Analyze this video for an AI training dataset.

        Find 5 to 8 distinct, high-energy, visually interesting moments
        (explosions, stunts, fast motion, money).

        Ignore static talking heads.

        Return a RAW JSON list.

        Format: [{"start": "MM:SS", "end": "MM:SS", "caption": "visual
        description"}]

        Constraints:

        1. Clips must be between 2 and 4 seconds long.

```

2. Do NOT use the name "MrBeast". Use "a man" or "a person".
3. Describe the visual style (e.g., "cinematic lighting", "wide shot", "color graded").
4. Output ONLY valid JSON. No markdown formatting.

"""

```
# *** UPDATED TO YOUR AVAILABLE MODEL ***
```

```
model = genai.GenerativeModel(model_name="gemini-3-flash-preview")
```

```
response = model.generate_content(  
    [video_file, prompt],  
    request_options={"timeout": 600}  
)
```

```
# Clean up the response
```

```
text_resp = response.text
```

```
if "```json" in text_resp:
```

```
    text_resp = text_resp.replace("```json", "").replace("```", "")
```

```
text_resp = text_resp.strip()
```

```
# Parse JSON
```

```
clips = json.loads(text_resp)
```

```
# --- STEP 3: CUT VIDEO ---
```

```
print(f" [3/3] Cutting {len(clips)} clips...")
```

```
base_name = os.path.splitext(filename)[0]
```

```
for i, clip in enumerate(clips):
```

```
    # Helper to convert MM:SS to seconds
```

```
    def to_sec(ts):
```

```
        parts = list(map(int, ts.split(':')))
```

```
        return parts[0] * 60 + parts[1]
```

```
    start = to_sec(clip['start'])
```

```

end = to_sec(clip['end'])
duration = end - start

# Enforce max duration of 4 seconds
if duration > 4: duration = 4

# Define output filenames
out_name = f"{base_name}_{i}"
vid_out = os.path.join(OUTPUT_FOLDER, f"{out_name}.mp4")
txt_out = os.path.join(OUTPUT_FOLDER, f"{out_name}.txt")

# Run FFMPEG
try:
    (
        ffmpeg
        .input(video_path, ss=start, t=duration)
        .output(vid_out, an=None, vcodec='libx264', crf=18)
        .run(quiet=True, overwrite_output=True)
    )

    # Save the text caption
    with open(txt_out, "w", encoding="utf-8") as f:
        f.write(f"mrbeast style, {clip['caption']}")

    print(f"    -> Saved: {out_name}.mp4")

except ffmpeg.Error as e:
    print(f"    FFmpeg Error: {e}")

except Exception as e:
    print(f"    FAILED to process {filename}: {e}")

```

```

# --- MAIN EXECUTION ---

```

```
if __name__ == "__main__":
    if not os.path.exists(OUTPUT_FOLDER):
        os.makedirs(OUTPUT_FOLDER)
        print(f"Created output folder: {OUTPUT_FOLDER}")

    print(f"Looking for videos in: {INPUT_FOLDER}")
    print("Starting Auto-Director...")

    for file in os.listdir(INPUT_FOLDER):
        if file.lower().endswith(('mp4', 'mov', 'mkv', 'avi')):
            # Skip the output folder itself
            if file == "mr_beast_dataset": continue

            full_path = os.path.join(INPUT_FOLDER, file)
            analyze_and_cut(full_path, file)

            # Increased sleep time to 5 seconds to be safe with the Preview model
            time.sleep(5)

    print("\nDone! Check the 'mr_beast_dataset' folder.")
```

2. Cloud Infrastructure Setup

We deployed a cloud GPU instance to handle the compute-intensive training.

- **Provider:** RunPod.io
- **GPU:** NVIDIA RTX A6000 (48GB VRAM).
- **Storage Configuration (Critical):**
 - **Container Disk:** 40GB (For system dependencies).
 - **Volume Disk:** 80GB (Mounted at /workspace to store the 25GB model and dataset).

3. Environment Installation

Upon booting the Linux environment, we installed the specific AI dependencies and fixed version mismatches between PyTorch and Hugging Face libraries.

Terminal Commands:

1. System Tools

```
apt-get update && apt-get install -y ffmpeg libsm6 libxext6 unzip
```

2. Clone Diffusers Library (Source Code)

```
cd /workspace
```

```
git clone https://github.com/huggingface/diffusers.git
```

3. Install Python Dependencies

```
pip install git+https://github.com/huggingface/diffusers.git
```

```
pip install transformers accelerate peft bitsandbytes pandas
```

4. Version Patching (Crucial for CogVideoX)

```
# Upgrading Torch to 2.4 to resolve 'pytree' attribute errors
```

```
pip install torch==2.4.0 torchvision==0.19.0 torchaudio==2.4.0 --index-url  
https://download.pytorch.org/whl/cu121
```

```
pip install bitsandbytes==0.49.1 accelerate==0.33.0
```

4. Dataset Sanitation & Formatting

We uploaded the dataset zip file and performed two critical cleaning steps: removing corrupted video headers (which caused decord crashes) and formatting the file lists for the training script.

Step A: The "Dataset Doctor" (Corruption Removal)

```
# doctor.py
```

```
import os, decord
```

```
DATA_DIR = "/workspace/training_dataset"
```

```
for f in os.listdir(DATA_DIR):
```

```
    if f.endswith(".mp4"):
```

```
        try:
```

```
            # Attempt to read first frame
```

```
            decord.VideoReader(os.path.join(DATA_DIR, f))
```

```
        except:
```

```
print(f"Deleting corrupted file: {f}")
os.remove(os.path.join(DATA_DIR, f))
```

Step B: The Metadata Formatter

```
# fix_dataset.py
import os
DATA_DIR = "/workspace/training_dataset"
# Generates paired text files required by the script
with open("videos.txt", "w") as vf, open("prompts.txt", "w") as pf:
    for vid in sorted(os.listdir(DATA_DIR)):
        if vid.endswith(".mp4"):
            txt_path = vid.replace(".mp4", ".txt")
            if os.path.exists(os.path.join(DATA_DIR, txt_path)):
                vf.write(vid + "\n")
                with open(os.path.join(DATA_DIR, txt_path)) as t:
                    pf.write(t.read().strip().replace("\n", " ") + "\n")
```

5. The Training Execution

This is the core command that initiated the LoRA fine-tuning. We utilized **Gradient Checkpointing** to reduce VRAM usage from >48GB down to ~30GB, preventing Out-Of-Memory (OOM) crashes.

Launch Command:

```
code Bash
downloadcontent_copy
expand_less
# 1. Navigate to script directory
cd /workspace/diffusers/examples/cogvideo

# 2. Set Environment Variables
export HF_HOME="/workspace/huggingface" # Redirect downloads to large disk
export MODEL_NAME="THUDM/CogVideoX-5b"
export DATASET_PATH="/workspace/training_dataset"
export OUTPUT_DIR="/workspace/mrbeast_lora"
```

3. Start Accelerate

```
accelerate launch train_cogvideox_lora.py \  
  --pretrained_model_name_or_path=$MODEL_NAME \  
  --instance_data_root=$DATASET_PATH \  
  --caption_column="prompts.txt" \  
  --video_column="videos.txt" \  
  --validation_prompt="mrbeast style, a red car driving off a cliff, cinematic lighting" \  
  --validation_epochs=20 \  
  --train_batch_size=1 \  
  --gradient_accumulation_steps=4 \  
  --gradient_checkpointing \  
  --learning_rate=1e-4 \  
  --lr_scheduler="cosine" \  
  --lr_warmup_steps=200 \  
  --max_train_steps=1500 \  
  --checkpointing_steps=500 \  
  --mixed_precision="bf16" \  
  --output_dir=$OUTPUT_DIR \  
  --enable_slicing \  
  --enable_tiling
```

6. Inference

Once training reached Step 500, the model weights were saved.

- **Output File:** `pytorch_lora_weights.safetensors` (~200MB).
- **Deployment:** The file was downloaded locally and loaded into **ComfyUI**.
- **Workflow:**
 1. Load CogVideoX-5b (Base Checkpoint).
 2. Attach LoraLoader (Select `mrbeast_lora.safetensors` at strength 1.0).
 3. Prompt: *"mrbeast style, [Subject + Action]"*.

Training Visual Generative Models: A Case Study in Video Style Transfer with CogVideoX-5B

Author: Kristijan Peshevski

Date: January 2026

Subject: Generative AI, Computer Vision, LoRA Fine-Tuning

Abstract

The field of Generative AI has rapidly expanded from Text-to-Image (T2I) synthesis to the far more computationally intensive domain of Text-to-Video (T2V). This document details the end-to-end process of fine-tuning a state-of-the-art video generation model, **CogVideoX-5B**, to replicate a specific visual style ("MrBeast Style"). We explore the theoretical underpinnings of 3D Causal Variational Autoencoders (VAEs), the efficiency of Low-Rank Adaptation (LoRA), and the practical engineering challenges involved in curating datasets, managing cloud GPU infrastructure, and overcoming hardware constraints like Out-Of-Memory (OOM) errors. This case study serves as a reproducible guide for training large-scale video models on consumer and prosumer hardware.

Table of Contents

1. Introduction: The Shift to Video

2. Theoretical Architecture

- Diffusion Transformers (DiT)
- 3D Variational Autoencoders (3D VAE)
- Low-Rank Adaptation (LoRA)

3. Phase I: Data Engineering & curation

- The "Garbage In, Garbage Out" Principle
- Automated Scene Detection with Gemini 3 Flash
- Dataset Sanitization

4. Phase II: Infrastructure & Environment

- Hardware Selection (NVIDIA RTX A6000)
- Storage & Environment Dependencies

5. Phase III: The Training Process

- Hyperparameter Configuration
- Gradient Checkpointing & Memory Optimization
- Handling Corrupted Data (The "Decord" Issue)

6. Phase IV: Inference & Application

- ComfyUI Workflow

7. Conclusion

1. Introduction: The Shift to Video

While Large Language Models (LLMs) process sequential tokens of text, and Image Models generate static grids of pixels, Video Models introduce the complex fourth dimension: **Time**.

Training an AI to generate video is exponentially harder than images because the model must maintain **temporal consistency**. A car appearing in frame 1 must look like the same car in frame 24, while simultaneously obeying the laws of physics, lighting, and motion.

In this project, our objective was not to train a model from scratch (which requires thousands of GPUs), but to perform **Style Transfer**. We utilized **CogVideoX-5B**, an open-weights model with 5 billion parameters, and fine-tuned it on a dataset of high-energy YouTube content to learn a specific directorial style: fast cuts, saturated colors, and high-dynamic action.

2. Theoretical Architecture

To understand *why* we took specific steps, we must understand the brain of the model we modified.

2.1. Diffusion Transformers (DiT)

CogVideoX is a Diffusion Transformer. Unlike older models that used U-Net architectures (like Stable Diffusion 1.5), Transformers allow for better scalability and context understanding. The model works by taking "noise" (random static) and gradually denoising it to reveal a coherent video, guided by a text prompt.

2.2. The 3D Variational Autoencoder (3D VAE)

This is the most critical concept in video AI. A 5-second video at 24fps contains 120 images. If each image is 1080p, the raw data is massive. No GPU can process that raw pixel data efficiently.

To solve this, CogVideoX uses a **3D VAE**.

- **Compression:** It compresses the video not just spatially (shrinking the image), but temporally (shrinking time).
- **Latent Space:** It turns the video pixels into a mathematical representation called "Latents."
- **The Process:** We train the AI on these compressed latents. During inference, the VAE "decodes" the math back into pixels.

2.3. Low-Rank Adaptation (LoRA)

Fine-tuning a 5 Billion parameter model requires updating all 5 billion weights. This would require over 80GB of VRAM just for the gradients. We did not have access to an H100 cluster.

The Solution: LoRA.

LoRA freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture.

- Instead of retraining the whole brain, we effectively trained a "plug-in" or a "filter" that sits on top of the brain.
 - **Benefit:** This reduced our VRAM requirement from ~80GB to ~30GB, making training possible on a single NVIDIA RTX A6000.
-

3. Phase I: Data Engineering & Curation

The quality of the dataset dictates the quality of the model. We automated the role of a "Human Video Editor" using Python and Multimodal LLMs.

3.1. The "Auto-Director" Pipeline

We developed a custom Python pipeline (`prepare_dataset.py`) to process raw YouTube footage.

1. **Download:** We utilized `yt-dlp` to extract high-bitrate MP4 files. Audio was discarded as current video models are visually unimodal.
2. **Scene Detection Strategy:** Initially, we considered standard pixel-based scene detection (`scenedetect`). However, this tool lacks semantic understanding—it splits clips when the camera cuts, but doesn't know *what* is happening.
3. **The Pivot to Gemini 3 Flash:** We integrated Google's **Gemini 3 Flash Preview** via API. We fed the raw video to Gemini and prompted it to act as a "Director."
 - *The Prompt:* "Identify 5 to 8 high-energy, action-packed moments. Return JSON timestamps and visual descriptions."
 - *The Result:* This provided us with semantically significant clips (explosions, stunts) rather than random cuts.

3.2. Captioning

For a Text-to-Video model to learn, Image and Text must be paired perfectly. If the video shows a car exploding, but the text says "a car," the model learns poorly.

Gemini 3 Flash generated dense, descriptive captions (e.g., *"Cinematic wide shot, a red sports car driving off a cliff, dust rising, 4k lighting"*).

We appended a **Trigger Word** (mrbeast style) to every caption. This creates a specific neuron activation path, allowing us to toggle the style on or off during generation.

3.3. Formatting for CogVideoX

The CogVideoX training script is strict. It requires a specific file structure.

We wrote a helper script (fix_dataset.py) to generate two parallel files:

- videos.txt: A list of file paths (e.g., video_01.mp4).
 - prompts.txt: A corresponding list of captions.
This ensured the data loader could stream the video from the disk without loading the entire dataset into RAM.
-

4. Phase II: Infrastructure & Environment

Training video models requires specialized High-Performance Computing (HPC) environments.

4.1. Hardware Selection

We utilized **RunPod.io** to rent cloud GPUs.

- **GPU:** NVIDIA RTX A6000 (48GB VRAM).
- **Why 48GB?** The CogVideoX-5B model weights alone are ~10GB. The optimizer states and gradients take up significantly more. A standard consumer card (RTX 4090 with 24GB) would have suffered Out-Of-Memory (OOM) errors immediately.

4.2. Storage Architecture

We encountered a critical failure point regarding disk space. The default cloud container provided 20GB of storage.

- *The Bottleneck:* Model (25GB) + Dependencies (5GB) + Dataset (2GB) > 20GB.
- *The Fix:* We redeployed the instance with an **80GB Volume Disk** mounted at /workspace. We configured the HF_HOME environment variable to force Hugging Face to download models to this larger volume, preventing system crashes.

4.3. Software Stack

We deployed a Linux environment (Ubuntu) with:

- **Python 3.10:** For stability with PyTorch.
 - **PyTorch 2.4 + CUDA 12.1:** We had to manually upgrade from PyTorch 2.1 to 2.4 to resolve `AttributeError: _register_pytree_node` compatibility issues with the latest Hugging Face libraries.
 - **Diffusers Library:** The core framework for training diffusion models.
 - **Accelerate:** A library to handle mixed-precision training and GPU offloading.
-

5. Phase III: The Training Process

This was the execution phase where the LoRA weights were calculated.

5.1. The Command

We used the `train_cogvideox_lora.py` script. Key parameters included:

- `--mixed_precision="bf16"`: We used **Brain Float 16**. Unlike FP16, BF16 prevents numerical instability (exploding gradients) which is common in training large transformers.
- `--learning_rate=1e-4`: A standard rate for LoRA. Too high, and the model forgets physics; too low, and it doesn't learn the style.
- `--rank=64`: The dimension of the LoRA matrices.

5.2. The "Decord" Corruption & The Doctor Script

Midway through initialization, the training crashed with a `DECORDError`.

- *Diagnosis:* The video loading library (decord) encountered a corrupted MP4 file header in the dataset.
- *Resolution:* We wrote a custom diagnostic tool (`doctor.py`) that iterated through every video file, attempted to decode the first frame, and programmatically deleted any file that threw an exception. This "Dataset Doctor" saved the project from manual debugging.

5.3. Optimization: Gradient Checkpointing

Despite using an A6000 (48GB), we initially hit a CUDA Out-Of-Memory error.

- *The Problem:* Storing the "activations" (intermediate math results) for 50 video frames requires massive memory.
- *The Solution:* We enabled `--gradient_checkpointing`.
 - *How it works:* Instead of storing all intermediate calculations in VRAM, the GPU throws them away and **re-calculates** them on the fly during the backward pass.

- *Trade-off:* It slows down training by ~20%, but it reduced VRAM usage from >48GB to ~30GB, allowing the training to run.
-

6. Phase IV: Inference & Usage

The output of the training was a .safetensors file (approx. 200MB). This file is not a standalone app, but a set of mathematical weights.

6.1. The Workflow

To generate video, we utilize **ComfyUI**, a node-based interface for Generative AI.

1. **Load Checkpoint:** Load the base CogVideoX-5b model.
 2. **Load LoRA:** Connect our trained mrbeast_lora.safetensors.
 3. **Prompt:** Input text like *"mrbeast style, a truck driving through a wall of water"*.
 4. **Sampler:** The model creates noise and denoises it over 50 steps, guided by our LoRA to ensure the colors and motion match the training data.
-

7. Conclusion

This project demonstrated that training state-of-the-art Video AI models is no longer the exclusive domain of massive tech corporations. By combining **Cloud GPUs**, **Smart Data Engineering (Gemini)**, and **Memory Optimization Techniques (LoRA, Gradient Checkpointing)**, we successfully fine-tuned a 5-billion parameter model on a budget of under \$10.

The result is a portable, reusable AI adapter that can apply a specific visual direction to any text prompt, paving the way for automated video production pipelines and personalized media generation.